Problem Statement

In CyberCop 2.0, you will replace the array-based implementation with efficient data structures from Java Collections Framework. You will also add more use cases.

The key changes are:

- Add, Modify, or Delete a case
- Use data in TSV and CSV formats with additional columns

Adding, modifying and deleting cases will manipulate only the in-memory data structures and those changes will be lost after the program termination. You will not modify the data files in this homework.

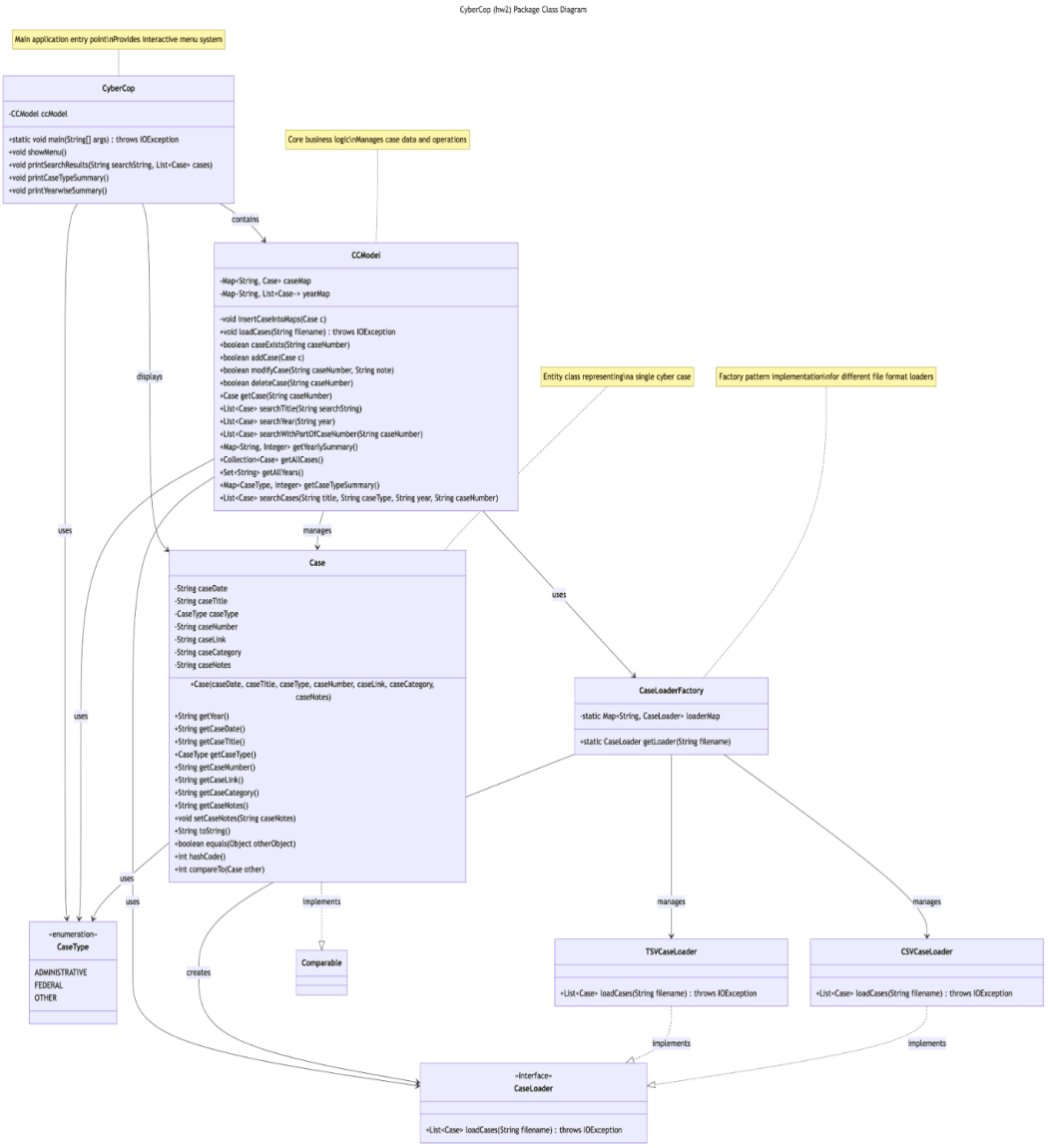
Solution Design

A class diagram for Cyber Cop 2.0 is provided in Figure. A brief description of each class is given below.

Caption	
Class Name	**Description**
CyberCop	It runs the user interactions
CCModel	This class is responsible for all data-related operations. Internal data structures of this class will be all private with no public getter methods for them. With encapsulation, no outside access of internal data structures will be possible. CyberCop will use this class via only its public methods.
Case	Abstraction for a case. Note: The caseType field is an enum of type CaseType. The constructor of Case class should take a String parameter for case type and then should set the caseType enum based on the text of the parameter.
CaseLoader	Interface with method loadCases
CaseLoaderFactory	A factory class that returns the appropriate loader object based on the extension of the data file, e.g. CSVCaseLoader object for .csv files, TSVCaseLoader object for .tsv files. This class should pre-populate the loader objects per data file type at the static initialization time and should not do type-based conditional at the run time.
CSVCaseLoader	It implements CaseLoader interface for .csv files
TSVCaseLoader	It implements CaseLoader interface for .tsv files

Table 1 Class Descriptions

See the UML diagram for details and for relations among classes.



SearchEngine class from Homework 1 will no longer exist in this homework. Its search functionalities will move to CCModel class.

Data

TSV Data set: A sample of this data set is shown in Table 2. It has case type (i.e., Federal, Administrative) extracted out into a separate column. It has three additional columns - Case link, Case category, and Case notes. However, not all rows may have data in these columns.

Caption						
Date	Title	Type	Case number	Case Link	Case Category	Case notes
2021-07-01	Kuuhuub, Inc., et al., U.S. v. (Recolor Oy)	Federal	1823184	https://www.ftc.gov/enforcement/cases-proceedings/1823184/kuuhuub-inc-et-al-us-v-recolor-oy	COPPA	Kuuhuub Inc., Kuu Hubb Oy and Recolor Oy settled FTC allegations that they violated a children s privacy law by ...
2021-05-07	Everalbum, Inc., In the Matter of	Administrative	192 3172	https://www.ftc.gov/enforcement/cases-proceedings/192-3003/support-king-llc-spyfonecom-matter	Facial recognition	Everalbum settled Federal Trade Commission allegations that it deceived consumers about its use of facial recognition technology...

Table 2 TSV Data Set

CSV Data set: This data has same columns as above, but since some of the values, such as Title or Case notes can have commas within them, they are enclosed within double-quotes as shown in Table 3.

Caption						
Date	Title	Type	Case number	Case Link	Case Category	Case notes
2021-07-01	"Kuuhuub, Inc., et al., U.S. v. (Recolor Oy) "	Federal	1823184	https://www.ftc.gov/enforcement/cases-proceedings/1823184/kuuhuub-inc-et-al-us-v-recolor-oy	COPPA	"Kuuhuub Inc., Kuu Hubb Oy and Recolor Oy settled FTC allegations that they violated a children s privacy law by ... "
2021-05-07	"Everalbum, Inc., In the Matter of"	Administrative	192 3172	https://www.ftc.gov/enforcement/cases-proceedings/192-3003/support-king-llc-spyfonecom-matter	Facial recognition	"Everalbum settled Federal Trade Commission allegations that it deceived consumers about its use of facial recognition technology..."

Table 3 CSV Data Set

Note TSV and CSV data sets are not identical.

For cases with no case number, your program should ignore them and not add them to the data structures. Note this is different behavior than Homework 1.

CyberCop 2.0 should be able to read data in both formats using polymorphism. You need to implement the functionality of loading the TSV data by parsing and splitting data on tabs. But for CSV data, you will use a 3rd party utility called CSVParser by including an external JARfile commons-csv-1.5.jar into your project. To use the jar file, you need to add it to your project. See the link below about how to use this jar file.

https://commons.apache.org/proper/commons-csv/apidocs/org/apache/commons/csv/CSVParser.htmlLinks to an external site.

Data Structures

You should choose the most efficient data structures from JCF to perform the following operations efficiently:

- Search with an exact case number: This will return the case with the case number
- Search with year: This will return a list of cases
- Delete a case: This will require the exact case number to delete the case
- Modify a case: This will require the exact case number to modify the case. Only the case notes will be modifiable.

These operations should execute in almost constant time or log(n) in the worst case. No efficiency requirements on other operations.

In your code, e.g. in CCModel.java, provide implementation comments about what data structures you choose and why.

Examine the supplied test cases file to learn about the methods to be implemented by the CCModel class.

User Interactions

You will add 6, 7 and 8 into the menu as below:

```
System.out.println("*** Welcome to CyberCop! ***");
System.out.println("1. Search cases for a company");
System.out.println("2. Search cases in a year");
System.out.println("3. Search case number");
System.out.println("4. Print Case-type Summary");
System.out.println("5. Print Year-wise Summary");
System.out.println("6: Add Case");
System.out.println("7. Modify Case Notes");
System.out.println("8. Delete Case");
System.out.println("9. Exit");
```

Choices 1-5 will provide the same user interactions as in Homework 1.

In this homework, run the menu until the user enters 9 or anything other than 1-8.

Add Case: Your program should ask for the case number from user and read it, then check if the case with that case number exists. If so, it should print a message "Case with the same case number already exists." and should go back to main menu. If not, then your program should ask the user to enter all other fields, create Case object and add it into the internal data structures.

Modify Case: Your program should ask for the case number from user and read it, then check if the case with that case number exists. If no case exists, then your program should print a message "Case with that case number does not exist." and should go back to main menu. If the case exists, then it should ask the user to enter case notes and update the case.

Delete Case: Your program should ask for the case number from user and read it. Then it should delete the case or print a message "Case not found. Deletion failed." if a case with the given case number does not exist.

Search Case: Similarly to Homework 1, with option 3 (Search case number), the user can enter part of a case number as opposed to an exact case number. Your program should search and print all cases that match the given partial case number. Since you will likely do a linear search for this use case, there is no efficiency requirement for it.

Note you will implement some methods in CCModel as seen in the UML diagram that are not exercised via User Interactions but they will the invoked by the supplied test cases.

Exception Handling

No need for exception handling in this assignment. You may assume the user will always enter valid inputs. However, your program should be robust to handle all user interactions and test cases without any crash/exceptions (i.e. your program should not throw null reference or other runtime exceptions) when executed with valid inputs.

Minimal Class Description

Implement the minimal class description requirements for the Case class by implementing the equals(), hashCode(), compareTo() (via Comparable interface) methods although these methods may not be invoked for this homework assignment. Implement these methods by comparing only caseNumber field and by generating hash code using it. Implement toString() for the Case class to return caseNumber only.

Backward Compatibility

No backward compatibility with Homework 1 is required. Since you will be ignoring the cases with no caseNumber in this homework, your output with respect to number of cases per type and per year would be different from Homework 1 output.

Packaging

Name your package as hw2 and put all your files under this package. Not doing so will lose you points.

Keep your data files at the project folder. Make sure your application and test cases run with the data files. Not putting the data files in the project folder may cause your test cases fail and may lose you points.

Downloads

Data:

CyberCop-TSVData.tsvDownload CyberCop-TSVData.tsv
CyberCop-CSVData.csvDownload CyberCop-CSVData.csv

Test Cases:

TestCyberCop2.javaDownload TestCyberCop2.java

Jar for CSV Processing:

commons-csv-1.5.jarDownload commons-csv-1.5.jar

Submission

Create and submit AndrewId-hw2.zip by adding only the Java code files. Test-file, data files, class files or any other files should not be included in the zip.

Only the last submission before the deadline will be graded. No late submission will be accepted.

Grading

- Test Cases: 12 points (0.8 points for each test case)
- Correct Execution and Correct Output: 8 points
- Rest: 8 points. The rest includes
    - Choosing efficient data structures and providing good documentation about them
    - Minimal class description for Case class
    - Overall documentation and comments for the whole code per Documentation and Commenting guidelines
    - Code quality and coding style
    - Correct packaging and submission